

Module 1

SystemVerilog and Verification Basics

Learning objectives

- Understand SystemVerilog for verification and verification fundamentals

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["SystemVerilog Classes and In"]

T2["Interfaces and Modports"]

T1 --> T2

T3["Packages and Namespaces"]

T2 --> T3

Understand SystemVerilog for verification and verification fundamentals

Overview

- This module establishes the foundation for verification work. You'll learn essential SystemVerilog...

Syllabus topics

1. SystemVerilog Classes and Inheritance for Verification (1/9)

- Object-Oriented Programming Basics
- Classes and objects
- Instance variables and methods
- Static variables and methods
- Encapsulation concepts
- Access modifiers (public, protected, local)

1. SystemVerilog Classes and Inheritance for Verification (2/9)

- Inheritance in SystemVerilog
- Single inheritance
- Method overriding
- `super` keyword usage
- Virtual methods and polymorphism
- Abstract classes

1. SystemVerilog Classes and Inheritance for Verification (3/9)

- Special Methods
- ``new()` constructor
- ``copy()` and ``clone()` methods
- ``compare()` and ``convert2string()` methods
- ``print()` and ``sprint()` methods
- Class Design Patterns for Verification

1. SystemVerilog Classes and Inheritance for Verification (4/9)

- Factory pattern basics
- Singleton pattern
- Builder pattern
- Strategy pattern
- Base ``Transaction`` class with static variables (``id_counter``)
- Instance variables (``id``, ``data``, ``timestamp``)

1. SystemVerilog Classes and Inheritance for Verification (5/9)

- Special methods: ``new()`, ``copy()`, ``compare()`, ``convert2string()`
- Inheritance: ``ReadTransaction`` and ``WriteTransaction`` inherit from ``Transaction``
- Method overriding: Child classes override ``convert2string()` method
- Using ``super`` to call parent class methods
- Purpose: Initialize object when created
- Called: Automatically via ``Transaction txn = new();``

1. SystemVerilog Classes and Inheritance for Verification (6/9)

- Default Parameters: ``name`` parameter has default value
- Purpose: Deep copy of transaction object
- Null Check: Always validates input before accessing
- Usage: ``txn_copy.copy(txn1);`` copies all fields
- Returns: ``1`` if equal, ``0`` if not equal
- Usage: ``if (txn1.compare(txn2)) $display("Match!");``

1. SystemVerilog Classes and Inheritance for Verification (7/9)

- Uses: ``$sprintf()` built-in function for formatting
- Format Specifiers: ``%0d`` (decimal), ``%0h`` (hexadecimal)
- Usage: ``$display("%s", txn.convert2string());``
- Purpose: Encapsulated access to private fields
- Usage: ``int id_val = txn.get_id();``
- Polymorphism: Derived class method called when using base class handle

1. SystemVerilog Classes and Inheritance for Verification (8/9)

- Base transaction creation and manipulation
- Derived transaction examples showing inheritance
- Copy and compare operations working correctly
- String conversion examples
- Static Variables: ``id_counter`` is shared across all instances
- Instance Variables: Each transaction has its own ``id``, ``data``, ``timestamp``

1. SystemVerilog Classes and Inheritance for Verification (9/9)

- Special Methods: Enable UVM-compatible behavior (copy, compare, string conversion)
- Inheritance: ``ReadTransaction`` and ``WriteTransaction`` extend base functionality
- Method Overriding: Child classes customize string representation
- Function Syntax: ``function <return_type> <name>(<args>); ... endfunction``
- Null Safety: Always check for null before accessing object fields
- Built-in Functions: ``$sformatf()``, ``$error()``, ``$display()`` used throughout

2. Interfaces and Modports (1/8)

- SystemVerilog Interfaces
- Interface definition
- Interface signals
- Interface methods
- Interface instantiation
- Modports

2. Interfaces and Modports (2/8)

- Modport definition
- Direction specification (input, output, inout)
- Modport usage in modules
- Multiple modports per interface
- Clocking Blocks
- Clocking block definition

2. Interfaces and Modports (3/8)

- Clocking block signals
- Synchronous signal access
- Clocking block in interfaces
- Verification-Specific Interfaces
- Driver interfaces
- Monitor interfaces

2. Interfaces and Modports (4/8)

- Scoreboard interfaces
- Agent interfaces
- Interface Definition: ``apb_if`` interface with signals and clocking block
- Modports: ``master_mp`` and ``slave_mp`` modports with different signal directions
- Clocking Block: ``cb`` clocking block for synchronous access
- Interface Usage: Module using interface with modport

2. Interfaces and Modports (5/8)

- Testbench Integration: Testbench connecting to interface
- Task Usage: Clock generation task, APB transaction tasks
- Purpose: Generate free-running clock signal
- Built-in Features: ``forever`` loop, delay operator ``#``, bitwise NOT ``~``
- Result: Clock with 10 time unit period (50% duty cycle)
- Event Control: ``@(posedge clk)`` waits for clock edge

2. Interfaces and Modports (6/8)

- Non-blocking Assignment: ``<=` for sequential logic
- Wait Loop: ``while (!condition) @(posedge clk)`
pattern`
- Blocking Assignment: ``=` for initialization
(combinational)
- Non-blocking Assignment: ``<=` for sequential logic
(clocked)
- Interface Signal Access: ``bus.signal_name`` accesses
interface signals
- Interfaces provide structured communication between
modules

2. Interfaces and Modports (7/8)

- Modports control signal direction and access
- Clocking blocks enable synchronous signal access
- Interfaces are essential for UVM testbenches
- Event Control: ``@(posedge clk)`` synchronizes to clock edges
- Assignment Types: ``=`` (blocking) vs ``<=`` (non-blocking)
- Wait Patterns: ``while (!condition) @(posedge clk)`` for handshaking

2. Interfaces and Modports (8/8)

- Built-in Operators: ``~`` (NOT), ``&&`` (AND), ``!``
(logical NOT)

3. Packages and Namespaces (1/7)

- SystemVerilog Packages
- Package definition
- Package items (classes, functions, types)
- Package import
- Package compilation order
- Namespaces

3. Packages and Namespaces (2/7)

- Package namespaces
- Resolving name conflicts
- Wildcard imports
- Explicit imports
- Verification Packages
- Common verification types

3. Packages and Namespaces (3/7)

- Utility functions
- Shared classes
- Configuration types
- Package Definition: ``verification_pkg`` with common types and functions
- Package Import: Using ``import verification_pkg::*``
- Name Resolution: Handling name conflicts

3. Packages and Namespaces (4/7)

- Package Organization: Structuring verification code
- Function Definitions: ``is_valid_address()``,
``calculate_checksum()`` functions
- Task Definitions: ``wait_for_condition()`` task with timeout
- Purpose: Validates address range (returns 1 if valid, 0 if invalid)
- Usage: ``if (is_valid_address(addr)) ...``
- Key Feature: ``automatic`` keyword makes function re-entrant

3. Packages and Namespaces (5/7)

- Purpose: Calculates XOR checksum of 32-bit data
- Returns: 8-bit checksum value
- Usage: ``logic [7:0] checksum =
 calculate_checksum(data);``
- Purpose: Wait for condition with timeout
- Key Features:
- ``ref`` parameter passes by reference

3. Packages and Namespaces (6/7)

- Contains delays (``#1``)
- Demonstrates task with timing control
- Packages organize related code (types, functions, tasks, constants)
- Import statements control namespace
- Functions in packages are reusable across modules
- ``automatic`` keyword enables re-entrant functions/tasks

3. Packages and Namespaces (7/7)

- Tasks can contain delays, functions cannot
- Packages improve code reusability
- Essential for UVM library organization

4. Data Structures for Verification (1/8)

- SystemVerilog Data Types
- ``bit`` and ``logic`` types
- ``int``, ``longint``, ``shortint``
- ``real`` and ``realtime``
- ``string`` type
- ``enum`` types

4. Data Structures for Verification (2/8)

- Arrays
- Fixed-size arrays
- Dynamic arrays
- Associative arrays
- Queues
- Array methods

4. Data Structures for Verification (3/8)

- Structures and Unions
- ``struct`` definition
- Packed and unpacked structures
- ``union`` types
- Structure methods
- Verification-Specific Data Structures

4. Data Structures for Verification (4/8)

- Transaction queues
- Coverage bins
- Scoreboard structures
- Configuration structures
- Dynamic Arrays: Transaction arrays that grow dynamically
- Queues: FIFO queues for transaction management

4. Data Structures for Verification (5/8)

- Associative Arrays: Indexed by transaction ID
- Structures: Complex data structures for verification
- Array Methods: Using built-in array methods
- Class Methods: Queue and scoreboard methods
(``push()``, ``pop()``, ``check()``)
- ``push()``: Wraps ``push_back()`` for FIFO enqueue
- ``pop()``: Wraps ``pop_front()`` for FIFO dequeue

4. Data Structures for Verification (6/8)

- ``size()`: Returns queue element count
- Usage: ``queue.push(txn); txn = queue.pop();``
- ``add_expected()`: Stores expected values
- ``check()`: Compares expected vs actual using ``exists()` method
- Usage: ``scoreboard.add_expected(1, data);
scoreboard.check(1);``
- ``sample_address()`: Records address access

4. Data Structures for Verification (7/8)

- ``get_coverage()`: Calculates coverage percentage
- Built-in Features: ``foreach`` loop, ``exists()`
method, type casting
- Dynamic arrays provide flexibility
- Queues are ideal for FIFO operations
- Associative arrays enable key-value lookups
- Structures organize related data

4. Data Structures for Verification (8/8)

- Array methods (``push_back()`, ``pop_front()`, ``size()`, ``exists()`) are essential
- Wrapper methods provide clean interfaces to built-in array operations
- ``foreach`` loops iterate over associative arrays efficiently

5. Error Handling and Logging (1/7)

- SystemVerilog Assertions
- Immediate assertions
- Concurrent assertions
- Assertion severity
- Assertion coverage
- UVM Reporting

5. Error Handling and Logging (2/7)

- UVM messaging system
- Severity levels (FATAL, ERROR, WARNING, INFO, DEBUG)
- Verbosity levels
- Message formatting
- Exception Handling
- Try-catch blocks (SystemVerilog 2017+)

5. Error Handling and Logging (3/7)

- Error propagation
- Graceful error handling
- Logging Patterns
- Structured logging
- Log file management
- Debug logging

5. Error Handling and Logging (4/7)

- Production logging
- UVM Reporting: Using ``uvm_report_info()`, ``uvm_report_error()`, etc.
- Assertions: Immediate and concurrent assertions
- Error Handling: Try-catch blocks for exception handling
- Logging Patterns: Structured logging for verification
- Retry Logic: Task with retry mechanism using ``retry_task()`

5. Error Handling and Logging (5/7)

- Exception Objects: Custom exception class with
 `convert2string()` method
- Constructor: Initializes exception with message and
 code
- `convert2string()`: Formats exception for display
- Usage: ``$display("%s", exc.convert2string());``
- Purpose: Retry operation with configurable attempts
- Built-in Functions Used:

5. Error Handling and Logging (6/7)

- ``$display()`: Print formatted messages
- ``$urandom_range()`: Generate random numbers for simulation
- Usage: ``comp.retry_task(3);`` retries up to 3 times
- Purpose: Demonstrate different message severity levels
- Usage: Shows structured logging patterns
- UVM reporting provides standardized messaging

5. Error Handling and Logging (7/7)

- Assertions catch design errors early
- Exception handling enables graceful failures
- Logging is essential for debugging
- Tasks can implement retry logic with delays
- Built-in functions like ``$urandom_range()`` are useful for simulation
- Custom exception classes provide structured error information

6. SystemVerilog Functions and Tasks

- See docs/MODULE1.md

Hands-on examples

Module 1 self-check

```
$ ./scripts/module1.sh --check
```

```
Module 1 self-check  
(run ./scripts/module1.sh when ready)
```

Exercise scaffold

```
./scripts/module1.sh --scaffold
```

Practice & assessment

Exercises

- Transaction Class Design
- Interface Design
- Package Organization
- Data Structure Usage
- Error Handling

Assessment checklist

- Can create SystemVerilog classes with inheritance
- Can design and use interfaces with modports
- Can organize code using packages
- Can use appropriate data structures
- Can implement error handling and logging
- Can write basic SystemVerilog testbenches

Summary & next steps

- Understand SystemVerilog for verification and verification fundamentals
- Complete module1/CHECKLIST.md
- Run `./scripts/module1.sh --check`
- Next: Next module in course